Designing a Scalable AI Development Pipeline

AI Coding Agent Workshop — Advanced Session (Claude + Codex + Gemini)

Goal: From running one kitchen to managing a multi-line factory

John Wu | Apr 2026

Today's Journey

Slot	What we'll do
Concepts	Agentic design thinking + governance boundaries
Live Demo	A multi-agent parallel-review pipeline in action
Recap & Q&A	Key takeaways + exercises to try later

> Today's keyword: **designing pipelines** — not operating every step yourself

Part 1: Concepts

From "Running a Restaurant" to "Running a Dev Pipeline"

Intermediate (last session)	Advanced (today)
Restaurant: how one kitchen runs	Factory: how multiple lines coordinate
Single Agent + rules	Multi-agent orchestration + project-level coordination
Prompt / Context / Harness	Agentic Workflow design

> Last time you learned to run one kitchen. Today you design the whole factory floor.

The Three Pillars: Advanced View

Pillar	Intermediate	Advanced
Prompt	Instruction tightness	Multi-agent dispatch
Context	Modular CLAUDE.md	Layered Playbook (roles / stacks)
Harness	Permissions + iteration	Hooks (hard blocks at the tool-call layer)

> From "knowing how to use it" to "knowing how to design it" — today we unpack each advanced move.

Vibe Coding: When to use it, when not to

Good for Vibe	Reach for more structure
Throwaway scripts / one-off scrapers	Perf-critical paths, security-sensitive code
Small standalone features, utilities	Cross-system integrations
Exploratory prototypes	Architectural decisions (the AI doesn't know your trade-offs)

> Vibe Coding isn't bad — **know when to let go, and when to pull in a spec to tighten things up**

Spec-Driven Development (SDD)

When Vibe isn't enough, write a spec first and let the AI build to it

Style	What you do	What the AI does
Vibe	Throw out a sentence	Guess what you meant
SDD	**Write a clear spec first**	**Build to the spec, precisely**

A spec should answer three questions:
What (what to build), **Constraints** (what not to do), **Done** (how we know it's finished).

> The clearer the spec, the more predictable and acceptance-ready the AI's output.
> The OpenSpec `proposal.md` you'll see later is exactly these three

Why You Need a Workflow Framework

From Prompt Engineering to Workflow Engineering

Pain point	You've all hit this
Context drift	After a long conversation the AI forgets earlier constraints and quality slips
Lost requirements	Requirements live only in chat; nothing structured gets captured
Hard to scale across teams	Personal prompt tricks don't generalize into team capability
Unstable execution	Large refactors or multi-file changes lack a reliable way to push forward

> The shift: from "how do I write the prompt" to "**how do I build the**

Agentic Workflow — You design the pipeline; AI runs the production line

Level	Metaphor	How it works
Traditional	Handcraft workshop	You write all the code
AI-assisted	You operate the machine	You do the work; AI helps
Agentic	**You design the pipeline**	**You design the flow; AI executes**

You = pipeline designer

> **You** design the flow (spec) and do quality acceptance (review)
> **AI** runs each station (plan → implement → test)

Multi-Agent Architecture — Why Split?

Problem: one Agent doing too much → quality drops

```
❌ One Agent doing frontend + backend + tests + a sprawling conversation
   → Context explodes → AI forgets earlier instructions → quality collapses
```

Solution: each subagent focuses on one thing

```
✅ Main Agent (production-line manager)
   ├── Agent A: frontend only, loads matching stack + skills
   ├── Agent B: backend only,  loads matching stack + skills
   └── Agent C: tests only,    loads matching skills
```

> Just like microservices: **single responsibility, clear interfaces**

The Core Benefits of Multi-Agent

Not just "parallel speedup" — it's context hygiene

Benefit	Why it matters
Lower context density	Each session carries only what it needs
Single responsibility	Each Agent stays focused on one task type
Clear integration points	Review and sign-off have defined boundaries

> Multi-agent is first about **context hygiene**; speed is the second-order benefit.

When Should You Go Multi-Agent?

Good fit	Poor fit
Task already has a plan with clear subproblems	Still brainstorming; problem isn't pinned down
Each subtask has clear inputs/outputs	Task is too small; splitting costs more than it saves
There are integration and review points	Subtasks share a lot of implicit context

> The right order: **plan first, then split into Agents.**

Multi-Model Collaboration: Claude Code Orchestrates Others

Through plugins, Claude Code can dispatch to other models.

Plugin	Dispatches to	Good for
codex-plugin-cc	OpenAI Codex	Code review, second opinions
cc-gemini-plugin	Google Gemini	Global analysis on large codebases

- My take: Codex is strong at planning, Claude at implementation, Gemini at wide-scope analysis.

> Different models have different strengths — combine them and you're steadier than going solo.

Multi-Model Division: Writer and Reviewer Must Differ

Stage	Writer	Reviewer
Plan	Codex	Claude
Coding	Claude	Codex
Global analysis	Gemini	Codex / Claude

> **The same model shouldn't write and review its own work** — switching models means switching perspectives.

How Subagents Get Dispatched

Two ways to dispatch

Way	How it triggers	Example
Automatic dispatch	Describe the task; main Claude picks the Agent	"Build a message board; split roles, work in parallel"
Manual assignment	Explicitly name the Agent and the model	"Have `code-reviewer` do the review"

> Each subagent has its own **context**; main Claude only sees the final report.

Subagents Can Also Pin a Model

Not just switching manually — decide the tier at dispatch time

```
Dispatch two subagents to work in parallel:

A (use opus):  check all prompts against the principles in prompt-best-practices.md
B (use haiku): align README.md's heading levels and bullet style with the other READMEs in the playbook
```

> Decide each station's model tier when you design the pipeline — not manually after the fact.

Agent Handoff Contract

Spell out what each subagent takes in and hands back

Field	What to define
Scope	Which slice of work it owns
Inputs / Outputs	What it receives; what it delivers
Write scope	Which files it may touch (e.g. reviewers are read-only)
Done	Acceptance criteria + what it must not do

> Without a handoff contract, you're not dividing work — you're distributing chaos

Config Layers: Project vs Personal

Type	Project (travels with the repo)	Personal (travels with you)
Prompts	`repo/CLAUDE.md`	`~/.claude/CLAUDE.md`
Settings	`repo/.claude/settings.json`	`~/.claude/settings.json`
Skills	`repo/.claude/skills/`	`~/.claude/skills/`

> Project-level is maintained by seniors; personal-level is yours to manage. On conflict, **project wins** — team rules beat personal preferences.

Why You Need a Playbook

One team, N projects — how do you keep standards from drifting?

Without a Playbook	With a Playbook
Every repo writes its own CLAUDE.md	Share one set, pulled in as a submodule
New hires relearn rules per project	Learn one set, understand every project
Rule updates pushed to N repos by hand	Update the Playbook, `pull` in each repo

> Playbook = the team's shared AI working standards, rolled out across every project at once.

Playbook — Shared Standards Across Projects

Managed uniformly via `submodule`

- playbook is **core + overlay**:
  `roles / stacks / policies / agents / hooks / skills / templates / guides`
- Every project pulls the submodule → they all share the same standards.

> No need to rewrite in each repo — **maintain in one place, apply everywhere**

Playbook Layered Structure

Priority (high → low) + who wins on conflict

```
CLAUDE.md                ← global rules (highest)
  → policies/            ← team red lines — e.g. security, data classification, PR must be reviewed before merge
    → roles/             ← role perspective
      → stacks/          ← tech rules
        → skills/        ← working methods
          → templates/   ← I/O templates (lowest)
```

> When a Role conflicts with a Policy → Policy wins.
>
> When a Skill conflicts with a Stack constraint → Stack wins.

Permission Mode: How much do you trust the AI?

Mode	What the AI can do	Good for
default	Most actions need your approval	High-risk or unfamiliar tasks
acceptEdits	File edits auto-approved; other ops still ask	Everyday development
bypassPermissions	Skips all permission prompts	Automated flows with solid guardrails

> Trust level must match guardrail strength — **the looser the permissions, the more you need Hooks**

Common Agentic Failure Modes

Symptom	Root cause	Fix
Agent drifts off-plan	Spec wasn't specific enough	Tighten the spec's constraints
Subagents produce conflicting output	No shared interface agreement	Define the API contract before splitting
Gets slower, quality declines	Context polluted by earlier errors	New session with a clean set of files

> Most Agentic failures aren't the AI being dumb — they're **the spec or the context going wrong**.

Failure Isolation & Rollback — Contain failures at each station

Practice	Purpose
Each subagent runs in its own context	Memories don't cross-contaminate (pair with `git worktree` if needed)
Pause for review before side effects	Block irreversible actions — MRs, DB changes
On error, restart from a clean session	Stops dirty context from piling up

> Advanced isn't about avoiding errors — it's making errors **isolated, reversible, replayable**.

Part 2: Live Demo — Multi-Agent Pipeline

Main Claude implements + multiple reviewers audit in parallel

Load the Playbook

Initialize the project

```
mkdir live-demo && cd live-demo && git init
git submodule add https://github.com/johnwu1114/playbook.git
./playbook/setup.sh
```

> One command drops all three pillars — Prompt, Context, Harness — into place
>
> ⚠️ First `claude` run triggers a `@playbook/CLAUDE.md` approval dialog — **be sure to approve**. If you accidentally deny, the import is disabled and you'll need to re-enable it manually.

State the Requirement

Prompt to Claude

```
Build a message board — frontend, backend, and a database running via docker.
Design it, split into three roles developing in parallel, then integrate and show it running.

(Don't ask me about specs item by item. Don't load any skill.)
```

> No stack specified — `@playbook/CLAUDE.md` decides which stack / policy overlays to load.

Dispatch 4 Playbook Reviewers in Parallel

Prompt to Claude

```
Dispatch code-reviewer / qa-checker / security-reviewer / migration-reviewer in parallel
to review what we just produced.
```

Multi-Model Collaboration

Prompt to Claude

```
Have codex run an adversarial-review.
If codex recommends cross-file impact analysis, dispatch gemini to back it up.
```

When Hooks Fire

Key: hooks are wired to Claude's tool calls

`policies/agent-security.md` goes into the system prompt to make the AI self-regulate (soft) — in practice the AI won't even try these commands in a demo.
Hooks are the last hard line: they only block when the AI actually tries to execute (hard).

```
cat > .env <<< "OPENAI_KEY=sk-sample"   # blocked by secret-guard
rm -rf ~/sample                         # blocked by dangerous-bash
```

> Policy (prompt layer, AI self-regulates) + Hook (tool-call layer, hard block)
> **Both layers together — that's defense in depth**

Demo Recap — Each Step Maps to a Pillar

Demo step	What happened	Pillar
Load the Playbook	One setup line pulls in roles/stacks/policies	**Context**
State the requirement	Three roles implement in parallel (frontend / backend / DB)	**Prompt**
Parallel review	4 reviewers in parallel, different angles	**Prompt**
Multi-model	Codex runs the adversarial review; escalate to Gemini if needed	**Cross-model**
When Hooks fire	Policy (prompt layer) + hook (tool-call layer)	**Harness**

Main Claude isn't going it alone

Playbook brings the knowledge, Agents divide the work, reviewers bring

Part 3: Exercises

Apply today's methods to your own work

Exercises — Build Your Own Review Pipeline

1. **Try it on a real MR**
   `code-reviewer` + `qa-checker` + `security-reviewer`
2. **Compare the reports**
   Who raises "cross-validation suggestions"? Any dimensions that overlap?
3. **Add your own Agent**
   e.g. `i18n-reviewer` / `perf-reviewer` / `api-contract-reviewer`
4. **Add it to the team Playbook**
   A good Agent is a team asset, not a personal preference

> Pipelines compound over time — every new station adds one more layer of scrutiny

Two-Week Rollout Path

Week 1	Week 2
Run MRs through the built-in reviewers	Add one custom reviewer
Note which suggestions repeat	Drop it in the team playbook; invite others to try
Log common false positives	Collect feedback; tune the prompt

> Don't redesign the whole pipeline at once — **start with one station, validate, then expand**

Part 4: Recap

How Do You Know the Pipeline Is Improving? — Agentic Metrics

Metric	What it's telling you
First-pass rate	Dropping usually means the spec or contract isn't clear
Review rejection rate	Rising means delivery quality is unsteady or acceptance lands too late
Human interventions	Rising means the automation boundary still isn't trustworthy
Lead time	Getting longer means too many handoffs or integration is stuck
Token / task	Rising usually means context pollution or too-fine splits; **especially watch multi-agent + multi-model flows**
Rollback count	Rising means risk controls placed too late in the flow

Multi-Agent Pipeline: Benefits & Costs

Dimension	What it means
Context hygiene	Each reviewer has its own context — not polluted by implementation chatter
Multi-dimensional quality	code / qa / security each bring their own lens — angles don't overlap
Parallel time savings	Three stations running at once — 2–3× faster than serial
Team asset accumulation	Agent `.md` files can be shared, versioned, reused across projects
Maintenance cost	Agent prompts must evolve with team rules — not write-once-and-forget
Decision authority	Reviewers advise; final decision still rests with main

Key Takeaways

1. The writer and the reviewer are never the same

Main Agent implements; reviewer Agents audit — independent context avoids self-bias.

2. Each station's prompt / model / tools are designed for its role

Sonnet reviews logic, Haiku runs spot-checks, **Gemini uses its 1M context for cross-file impact analysis**; reviewers get read-only tools.

3. Pipeline extensibility comes from file-based Agents

One `.md` file = one new station; the team playbook turns them into shared assets.

Three-Session Workshop Recap

Session	What You Learned
Beginner	Experience AI collaboration; complete your first sign-off-ready collaboration
Intermediate	Design the AI work environment so results are repeatable
Advanced	Design a dev pipeline so AI runs the process for you

> From "I use AI" → "AI works reliably" → "AI runs the process for me"

Q&A